



µLab worksheet • Kubernetes in one container

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

Hand this in at the end of the round. The referee presents this work in 90 seconds at the caucus, so the referee has to follow it, not just watch it.

1 • START THE CLUSTER, THEN GET A CLIENT • STEPS 1 AND 2

1 ☐ Create a container that is a Kubernetes cluster

```
mkdir kube
docker run -d --name k8s --privileged --tmpfs /run --tmpfs /var/run \
  -p 6443:6443 -v "$PWD/kube:/output" \
  rancher/k3s:v1.36.4-k3s1 server --disable=traefik \
  --write-kubeconfig /output/config --write-kubeconfig-mode 644
```

-d Detached. It runs in the background and you get your prompt back.
--name k8s A name of your choosing. Without it Docker still identifies the container, but by its id: 8d7726f96f747554440565a92a034cda1f767726619081595bab18cdf76987. Every later command would have to carry that, or its short form 8d7726f96f74.

--privileged Lifts the usual restrictions. k3s mounts cgroups and runs **its own** container runtime inside this container.
--tmpfs /run Two small writable in-memory filesystems k3s needs (also /var/run).
-p 6443:6443 **HOST:CONTAINER.** Port 6443 on your laptop is forwarded to 6443 inside the container. 6443 is where the cluster's **API server** listens, and step 2 is going to talk to it.

-v "\$PWD/kube:/output" Bind-mounts your new kube folder into the container, so a file written there lands on your laptop.
rancher/k3s:v1.36.4-k3s1 The image. rancher/k3s is the repository, v1.36.4-k3s1 the tag. You named no registry, so Docker went to **Docker Hub**.

server **Not a docker flag.** Everything after the image name is the command handed to the program inside. server tells k3s to be the cluster.

--disable=traefik Also k3s's. Traefik is an **ingress controller**: it routes HTTP arriving from outside to services inside. It claims ports 80 and 443 and adds startup time; nothing today arrives from outside.

--write-kubeconfig Writes the cluster's address and credentials to /output/config, which is your kube/config. -mode 644 makes it readable without sudo.

After a few seconds kube/config exists on your laptop. Open it. One line reads server: https://127.0.0.1:6443 — that is the only address anything will need.

2 ☐ Get a kubectl, and point it at that address

kubectl is **not part of the cluster**. It is an ordinary program that sends HTTPS requests to the address above, so it can live anywhere. Pick one.

(a) On Linux — install it on your laptop

```
sudo snap install kubectl --classic
export KUBECONFIG=$PWD/kube/config
```

It reaches the cluster through the port you published in step 1. This is exactly how you would drive a real cluster from your own machine.

(b) Anywhere, including macOS and Windows — run kubectl in its own container

```
docker run --rm -it --network container:k8s \
  -v "$PWD/kube:/kube:ro" -e KUBECONFIG=/kube/config \
  --entrypoint sh alpine/kubectl
```

--rm Delete this container the moment you leave it. The cluster in step 1 used -d because it must outlive your prompt; this one is disposable and you can start another whenever you like.

-it -i keeps the input open, -t gives a terminal. Together: an interactive shell instead of a single command.
--network container:k8s Do not give this container its own network — **join the one the cluster container already has**. That is why 127.0.0.1:6443 works in here: it is the cluster's own loopback, which is exactly the address written in kube/config.

-v "\$PWD/kube:/kube:ro" Bind-mount again, the other way round: your kube folder appears inside this container at /kube. :ro makes it **read-only** — a client has no business editing the cluster's credentials.

```
-e KUBECONFIG=/kube/config Sets an environment variable inside the container. kubectl reads KUBECONFIG to find out which cluster to talk to. Note the path is the one inside this container, not the one on your laptop.
--entrypoint sh Overrides what the image runs by default. alpine/kubectl is built to run kubectl and nothing else, so without this you would get one command per docker run. Swapping in a shell lets you type many.
alpine/kubectl The image: kubectl and a small Alpine base, nothing else. No registry named, so Docker Hub again.
```

From this prompt, commands are typed as plain `kubectl`, with no `docker` in front.

There are now **two containers running**, and a second terminal shows both:

```
docker ps
NAME          IMAGE              STATUS
client        alpine/kubectl     Up 2 seconds
k8s           rancher/k3s:v1.36.4-k3s1 Up 4 minutes
```

One is the cluster, one is the client. Leaving the client stops only the client: `--rm` deletes it, and the cluster carries on without it.

Either way, you are outside the cluster.

Nothing from here on is typed **inside** the cluster. The cluster is one program and the client is another, and the only thing joining them is the address in `kube/config`. Which did you pick, (a) or (b)? _____

Now check that it reaches the cluster.

`kubectl get nodes` asks the API server for the machines that make up the cluster. There will be exactly one: the container you started in step 1. If this prints a table, your client found the cluster and everything after this will work.

```
kubectl get nodes
```

Expect something close to:

NAME	STATUS	ROLES	AGE	VERSION
7c19eb7874a1	Ready	control-plane	11s	v1.36.4+k3s1

Instead got `The connection to the server localhost:8080 was refused?` Then `kubectl` never found the config and fell back to a built-in default. Check that `kube/config` exists and is not empty, and that you are running from the folder that contains `kube`.

Write down what that table said.

This is the cluster from step 1, described by the cluster itself. The figures come back in panel 4.

NAME	_____	VERSION	_____
STATUS	_____	AGE	_____

2 • SOME DEFINITIONS

Step 1 produced a Kubernetes cluster inside one container. Step 2 produced a `kubectl` outside it that can reach the cluster's API. Steps 3 to 6 use the words below, and so does every question in panels 4 to 9. Come back here.

- kubectl** The cluster's command-line client. It runs nothing itself: every command becomes an authenticated HTTPS request to the cluster's API, and the reply is printed.
On an HPC cluster running **Slurm**, work is submitted by first logging in to a **login node** and running `sbatch` there. Kubernetes has no login node. `kubectl` carries the credentials from `kube/config` and authenticates each request on its own, so the machine it runs on is doing the login node's job.
- pod** **A running container, as Kubernetes counts it.** `docker run` produces a container; Kubernetes produces a pod, with the container inside it. A pod may hold several containers that have to share one machine and one IP address, but holding a single container is the ordinary case. **Pods are never created directly.**
- Deployment** **Kubernetes' object for a service: something meant to be up.** It is a written request, stored by the cluster, of the form *keep N pods of image X running*. Creating one starts no container: the request is sent, the cluster saves it, and the command returns. The pods appear a moment later, started by something else.
Why request it rather than start it? On a real cluster there are many machines, and picking one by hand would mean tracking which has room. Worse, machines and processes fail: a container started by hand stays dead, while a request that is stored can go on being satisfied after the failure.
- Job** **Kubernetes' object for a task: something meant to finish.** The stored request reads *run image X once, until it exits successfully*. One pod is started, its exit is waited for, and nothing is replaced.
- restartPolicy** **The field that says what to do when a container stops** — and the one field that separates the two objects above. A Deployment's pods get `Always` a Job's pods get `Never`. Neither is ever typed: the kind that was asked for fills it in.
`Always` does **not** mean *on failure*. It means *whenever this container is not running, start it*. The exit code is not consulted, because a service that has stopped is not serving, whatever it thought it was doing.
Panel 4 reads this field off a running pod.
- controller** **A program inside the cluster that reads the stored requests and changes the cluster until they are met.** Take a Deployment that says three pods. The controller counts the pods that exist. Two, so it starts one. Four, and it would stop one. Then it counts again, and it keeps counting for as long as the request is stored.

	This is why a deleted pod comes back: deleting a pod changes the count, not the request. Note what a controller works on: whole pods.
kubelet	The part of Kubernetes that runs on each machine and does the actual starting and stopping of containers there. It works one level below the controller: not on pods, but on the containers inside a pod that already exists. When a container stops, the kubelet is what consults <code>restartPolicy</code> and starts it again — same pod, same name, same node. So there are two loops, and they are easy to confuse. The kubelet restarts a container in place, and RESTARTS goes up. A controller replaces a whole pod , and a new name appears with RESTARTS back at 0. Panel 4 is the first, panel 9 the second.
scheduler	The program that decides which machine a new pod goes to. With one machine there is nothing to decide, but it still runs and still says so. It appears in panel 7 under the name <code>default-scheduler</code> .

3 · DRIVE THE CLUSTER · STEPS 3 TO 6

Type these with the `kubect`l set up in step 2.

3 ☐ Create a Deployment

The aim of this round is to watch what a cluster does when a container stops. So the object to create is a Deployment: a stored request for three pods that should be running. Three rather than one, only so that there is more to watch.

Read step 4 first. What happens next happens in seconds, and step 4 is how to see it. Nothing is lost by being slow, but the first few seconds are the interesting ones.

```
kubectl create deployment whale --image=ghcr.io/oesteban/whalesay --replicas=3
```

create	The verb: make a new object. Others are <code>get</code> , <code>describe</code> , <code>scale</code> , <code>delete</code> , <code>logs</code> .
deployment	The kind of object to create. <code>job</code> , <code>pod</code> , <code>service</code> are other kinds.
whale	The name , chosen freely. Every later command refers to it by this name. It also becomes the label <code>app=whale</code> and the prefix of every pod name it creates, which is why the pods come out called <code>whale-544548b554-kpnfx</code> .
--image= ...	Which image the pods run. The only part the cluster has to fetch from a registry.
--replicas=3	How many pods the request asks for. Leave it out and the default is 1.

Why `ghcr.io/...` spelled out, when step 1 named no registry at all? There is a second copy of this image on Docker Hub, and it does not work on every laptop. Panel 11, the annex, has the answer and a command to check it.

4 ☐ Watch what the cluster does with it

`kubectl get pods` prints the list once and returns, which is no use for something that changes every few seconds. `watch` runs a command over and over and redraws the screen in place, so the table stays three lines long and the columns change under the eye.

```
watch kubectl get pods
```

watch	Re-runs whatever follows it, every two seconds, redrawing rather than scrolling. <code>-n 1</code> makes it every second. Ctrl-C ends it. Nothing to do with <code>kubect</code> l: it is an ordinary command-line tool.
get	List objects, one line each. No name given, so it lists them all.
pods	The kind to list. <code>pod</code> , <code>pods</code> and the short <code>po</code> all work. <code>kubect</code> l <code>get deployments</code> and <code>kubect</code> l <code>get jobs</code> list the other two kinds.

Three pods, one line each, redrawn every two seconds:

```
Every 2.0s: kubectl get pods                2026-09-21 09:38:01
```

NAME	READY	STATUS	RESTARTS	AGE
whale-544548b554-lbrnj	0/1	Completed	2 (21s ago)	23s
whale-544548b554-pbc2b	0/1	Completed	2 (21s ago)	23s
whale-544548b554-xrkvc	0/1	Completed	2 (21s ago)	23s

Watch for two minutes. **STATUS** flickers between `Running`, `Completed` and `CrashLoopBackOff` — the container only runs for a fraction of a second, so `Running` is rarely caught. **RESTARTS** is the column that tells the story: it only ever goes up. Record an early line and a later one in panel 4, then **Ctrl-C**.

CrashLoopBackOff is not a problem here, and nothing needs fixing. It is the expected result: a container that finishes is being asked to stay running, so it is started again and again, with a growing pause in between. That pause is what the status is reporting.

To run the whole thing again from the start: delete the Deployment, then repeat step 3.

```
kubectl delete deployment whale
```

`delete` removes the stored request, and its pods go with it.

5 ☐ Create a Job from the same image

A Job is the other kind of stored request: run the image once, until it finishes. The image and the cluster are the same as in step 3, and only the kind is different — so anything that happens differently is caused by the kind alone.

```
kubectl create job whale --image=ghcr.io/oesteban/whalesay -- cowsay 'a cluster ran me'
```

job	The kind. Everything else on this line works exactly as it did in step 3.
--	Ends kubect l's own arguments. What follows is not for <code>kubect</code> l: it is the command to run inside the container, and it replaces the one built into the image. The same idea as <code>--entrypoint</code> in step 2, and of <code>ENTRYPOINT</code> in a Dockerfile.

`cowsay 'a cluster ran me'` That command and its single argument. `cowsay` is the program inside the image; the quoted text is what it prints.

Then `kubectl get pods` once more. The Job's pod is the one whose name carries a single hash rather than two.

A name can only be used once. Running that same line a second time fails:

```
error: failed to create job: jobs.batch "whale" already exists
```

To run another one, either give it a different name, or delete the first:

```
kubectl create job whale2 --image=ghcr.io/oesteban/whalesay -- cowsay 'again'
kubectl delete job whale
```

Note that the Deployment from step 3 is **also** called `whale`, and it was not in the way. Names have to be unique within a kind, not across kinds — which is why the next step has to say `job/whale` and not just `whale`.

6 ☐ Read what the Job printed

The pod ran `cowsay` and exited. What it printed is still held by the cluster, and this is how to see it.

```
kubectl logs job/whale
```

`logs` Print what a container wrote to its output. Containers have no screen; this is the screen.
`job/whale` **KIND/NAME.** A Job does not have logs — its **pod** does. Naming the Job is enough: `kubectl` finds the pod it made. `deploy/whale` works the same way, and a bare pod name also works.

4 • WHY THE DEPLOYMENT KEPT RESTARTING

Two lines from the watch in step 4.

One copied in the first seconds, one copied about a minute later.

An early line

STATUS _____

RESTARTS _____

A later line

STATUS _____

RESTARTS _____

First, where those pods are running.

In step 2 the node came out with a name like `7c19eb7874a1`. Run `docker ps` on the laptop and read the `CONTAINER ID` column: that node name is the id of the container started in step 1. A real cluster has many machines and has to choose one; this cluster has a single container acting as its one machine, and the three pods are processes inside it.

Second, notice what did not change.

Compare the two lines written above. `RESTARTS` climbed — did the pod **NAME** change? It did not, so nothing was replaced: the same pod stayed where it was and the container inside it was started again. That is the **kubelet** in panel 2, not a controller. Panel 9 is about the other loop, where a whole pod really is replaced.

Third, read the one field that decides all of this.

`-o jsonpath=...` prints a single field of an object instead of the whole table; `.spec.restartPolicy` is the path to the field inside it. Put the same question to a pod of each kind:

```
kubectl get pod <one of the three> -o jsonpath='{.spec.restartPolicy}'
```

```
kubectl get pod <the Job's pod> -o jsonpath='{.spec.restartPolicy}'
```

Deployment's pod _____ Job's pod _____

Neither value was typed in step 3 or step 5. The kind that was asked for put it there.

Now the question this round is about.

`cowsay` printed its whale and exited. It did not crash: it finished, which for a program is the ordinary way to end. The cluster starts it again anyway, and `RESTARTS` keeps climbing.

Using the two values just written down and the entry for `restartPolicy` in panel 2, say why.

The Job in step 5 ran **the same image** on **the same cluster** and was started exactly once. The image did not change and the cluster did not change, so the difference is something you wrote. What was it, and which field did it set for you?

On CrashLoopBackOff again: it is the status shown while the cluster waits before starting a stopped container one more time. The wait grows each round, 10s, 20s, 40s, doubling up to a five-minute ceiling, which is why the table slows down the longer it is watched. Nothing has crashed and nothing needs fixing.

Then read the cluster's own account of it.

`get` prints one line per object. `describe` prints everything the cluster knows about **one** object, which for a pod is some fifty lines.

`kubectl describe pod whale-544548b554-528fl`

Substitute one of your own pod names from `kubectl get pods` the random part differs on every laptop.

About a third of the way down sits a block headed **Last State**. It describes the previous run of the container, the one that has already ended. Copy two of its fields:

Reason _____ Exit Code _____

Then go to the very bottom of the same output, to the block headed **Events**. One of its lines reads:

Warning BackOff ... Back-off restarting failed container whalesay

Those two parts of one output describe the same container and do not agree about it: one says **Completed**, the other says **failed**.

Panel 2 defines a Deployment as the object for something **meant to be up**. Read from there: which of the two words is wrong, and from a Deployment's point of view, is it wrong at all?

5 • READING LOGS

A container has no screen. Whatever the program inside writes is kept by the cluster, and `kubectl logs` prints it back. Type these with the `kubectl` from step 2, while the Deployment from step 3 still exists.

`kubectl logs <pod>`

`kubectl logs deploy/whale`

The plain form. Take a pod name from `kubectl get pods`. Out comes the whale.

KIND/NAME, as in step 6. Logs belong to a container and the Deployment has three, so `kubectl` picks one and admits it on the first line: Found 3 pods, using pod/whale-.... Run it twice: the same pod answers both times.

Now suppose only one of the three pods is misbehaving. What is the risk in letting `kubectl` choose for you?

`kubectl logs -l app=whale --prefix`

All three at once. `-l` selects by **label** rather than by name, and step 3 stamped `app=whale` on every pod it created. `--prefix` puts [pod/NAME/CONTAINER] at the start of each line.

Run it once with `--prefix` and once without. Three whales arrive either way. Name a question the prefixed output answers and the bare one cannot.

`kubectl logs <pod> --timestamps`

Puts the time each line was written in front of it. Subtract the first timestamp from the last: that is how long the program took, and the interesting digits are after the decimal point.

Compare it with the two seconds between redraws in step 4. Why was `Running` never on screen?

`kubectl logs <pod> --previous`

The run **before** the current one. Plain `logs` only ever shows the container running now, so once a container is replaced its output is out of reach without this. Here it often answers unable to retrieve container logs instead: these pods restart every few seconds and the dead container has already been cleared away.

So the cluster keeps the last run, sometimes, for a while. If a container's output were the only record of why it stopped, what would that mean for a cluster you actually depended on?

Answer that last one here. It is the reason log collection exists.

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

6 • RUN THE IMAGE YOU BUILT ON FRIDAY

Everything so far ran an image belonging to someone else. Now do step 3 again with your own: the one your GitHub Actions workflow pushed on Friday. Replace `<your-handle>` with your GitHub username.

```
kubectl create deployment mine --image=ghcr.io/<your-handle>/whalesay
```

No `--replicas` this time, so the default applies and there is one pod. It will restart for ever, for the reason written in panel 4. That is expected. What is being tested here is only whether the image arrives.

Your handle _____ Did the pod start? _____

If the image never arrived, STATUS reads `ErrImagePull` or `ImagePullBackOff`, and `kubectl describe pod` carries the registry's own message in the **Events** block at the bottom. Write that message down, then fix it.

A registry is a server that stores images and hands them out over HTTPS. Nobody told the cluster where `ghcr.io` is, and nobody gave it a password, yet it fetched the image.

Open your package's page on GitHub — your profile, then **Packages**. One setting there is the answer. Which, and what would the pod have done if it were set the other way?

`docker pull alpine` names no registry at all and still gets an image. A full image name has three parts before the tag: **registry / namespace / repository**. `ghcr.io/oesteban/whalesay` has all three written out.

Write `alpine` out the same way. Panel 8 lists image names in full, and the `rancher` ones there show the pattern.

7 • READ THE CLUSTER'S EVENT LOG

Every time the cluster decides something it writes the decision down: this pod goes on that node, this image was pulled and took 510 ms, this container was started, this one is being held back before another try. Those records are called **Events**. They are objects like pods, so `kubectl get` lists them.

```
kubectl get events --sort-by=.lastTimestamp | tail -8
```

<code>events</code>	The kind to list, exactly as <code> pods </code> was in step 4. This covers the whole cluster, not one pod.
<code>--sort-by=.lastTimestamp</code>	Order by a field of the object, oldest first. Without it the order is arbitrary and the list is unreadable.
<code> tail -8</code>	Ordinary shell, nothing to do with kubectl: keep the last eight lines, which are now the eight most recent events.

The same events, narrowed to one pod, are the block at the bottom of the `kubectl describe pod` output read in panel 4.

The **From** column names who wrote each line, and both names are defined in panel 2: `default-scheduler` chose the machine, `kubelet` did the work on it. The `BackOff` lines are the `kubelet`'s, which is consistent with panel 4.

Nobody typed any of these lines. Read them and say what they are a record of: the requests you made, or what the cluster did in order to meet them?

8 • THE CLUSTER HAS ITS OWN IMAGE STORE

k3s does not use the Docker daemon on the laptop. Inside the k8s container it runs a container runtime of its own, **containerd**, which keeps its own copy of every image it pulls. `crictl` is to containerd what `docker` is to Docker, and k3s ships it.

Both lines below are typed **on the laptop**, in a terminal that is not the `kubectl` one:

```
docker exec k8s crictl images      # the cluster's images
docker images                      # the laptop's images
```

`docker exec k8s ...` runs a command inside the k8s container. `crictl` exists only in there, so typing it on the laptop, or in the `kubectl` container, finds no such command.

whalesay in `crictl` ? _____ in `docker` ? _____

Nobody copied anything from the laptop into the cluster, and the two stores share no files. So where did the cluster's copy come from? One line of the event log in panel 7 says it outright, with a duration.

And what would step 3 have done if that image had only ever existed on your laptop?

9 • SCALE THE DEPLOYMENT, THEN DELETE A POD

A Deployment is a stored request, so changing how many pods run means editing the request. Nothing is started or stopped by hand.

```
kubectl scale deployment whale --replicas=5
kubectl scale deployment whale --replicas=0
```

`scale` Edits one field, `replicas`, of an object that already exists. It creates nothing.
`deployment whale` KIND then NAME, written with a space. The long form of `deploy/whale`.

Count the pods with `kubectl get pods` a few seconds after each line.

pods at 5 _____ pods at 0 _____

Now delete a pod that something is still asking for, and one that nothing is.

```
kubectl scale deployment whale --replicas=3
kubectl delete pod <one of the three>
kubectl delete pod <the Job's pod from step 5>
```

Then `kubectl get pods` again. One of the two is back within seconds and the other is simply gone. Which is which?

Panel 2 has the answer in two places: what a **controller** counts, and what each of the two stored requests actually asks for. Note that the replacement arrives with a **new name** and `RESTARTS` back at 0 — this is the second loop, not the kubelet's.

10 • EXTRA MILE • MAKE THE DEPLOYMENT STOP RESTARTING

Step 5 stopped the restarting by asking for a different **kind** of object. There is a second way, and it leaves the object a Deployment: same image, nothing rebuilt, nothing re-pushed, and the pods settle at `Running` with `RESTARTS 0`.

What you already know: the image runs one program and that program ends, and step 5 showed how to replace the program a container runs. Delete the Deployment, then create it again with one thing added.

Write the command, then run it.

There are now two ways to keep this cluster content with this image. Which one fits a web server, and which fits a report that is produced once a night?

11 · ANNEX · WHY THE IMAGE COMES FROM GHCR.IO AND NOT DOCKER HUB

Two copies of this image exist, `ghcr.io/oesteban/whalesay` and `oesteban/whalesay`, and they are not the same thing. Run these two *on the laptop*, not in the `kubectl` container:

```
docker buildx imagetools inspect ghcr.io/oesteban/whalesay
docker buildx imagetools inspect oesteban/whalesay
```

`imagetools inspect` asks a registry what a name points at, and prints the answer without downloading the image itself.

A name in a registry can point at a single image, or at a **list** of images with one entry per CPU architecture. The first command prints such a list, headed `Manifests:`, with `Platform: linux/amd64` **and** `Platform: linux/arm64`. The second prints no list at all: one image, `linux/amd64` only.

An Apple Silicon Mac runs an **arm64** Linux VM, and the `containerd` inside `k3s` has no emulation to fall back on. The Docker Hub copy would fail there with `exec format error` and the pod would never start. The GHCR copy was built for both architectures by GitHub Actions on Friday.

On Linux or Windows, on an Intel or AMD processor, the Docker Hub copy is fine. Check the manifests above, then run it alongside:

```
kubectl create deployment hub --image=oesteban/whalesay --replicas=1
```

Which architecture is this laptop? Ask Docker, not the shell — on a Mac the two answer differently, and it is Docker's Linux VM that runs the image:

```
docker version --format '{{.Server.Arch}}'
```

Architecture _____ Docker Hub copy runs? _____

Done when the referee can say, without reading this sheet, why the same image restarts forever as a Deployment and finishes as a Job.